



Unité de transmission/réception série asynchrone

Avertissement

Ce document est le document de référence. Certaines notations et schémas peuvent différer dans les transparents de présentation.

1 Introduction

Une UART, Universal Asynchronous Receiver/Transmitter, est un contrôleur d'entrées/sorties qui gère les ports série des ordinateurs. Il en existe différents modèles : les 8250 (8 bits), les 16450 (16 bits) et la famille des 16550 (16 bits + FIFO). Cette dernière version est utilisée dans tous les PCs actuels. L'UART, présente aussi sur les modem, permet la communication entre le modem et le PC. L'objectif est l'implantation d'une UART 8 bits simplifiée.

2 Description de l'UART

2.1 Interface

L'UART que nous allons développer possède l'interface présentée en FIGURE 1.

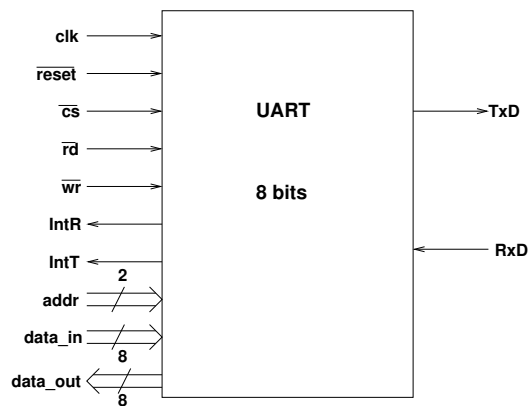


FIGURE 1 – Interface de l'UART

```
entity UARTunit is
  port ( clk , reset : in  std_logic ;
         cs , rd , wr : in  std_logic ;
         RxD          : in  std_logic ;
         TxD          : out std_logic ;
         IntR , IntT  : out std_logic ;
         addr         : in  std_logic_vector(1 downto 0);
         data_in      : in  std_logic_vector(7 downto 0);
         data_out     : out std_logic_vector(7 downto 0) );
end UARTunit ;
```

2.2 Spécifications

L'UART est composée d'un registre de contrôle `ctrlReg` et de 4 unités fonctionnelles :

- `clkUnit`, cette unité décompose `clk`, l'horloge de base du circuit (celle du processeur), en deux "horloges" nommées `enableTX` et `enableRX` qui contrôlent respectivement l'émission et la réception ;
- `ctrlUnit`, cette unité contrôle l'état de l'UART ;
- `TXUnit` est l'unité d'émission ;
- `RXUnit` est l'unité de réception.

La FIGURE 2 montre la spécification interne du circuit UART (attention : toutes les connexions ne sont pas représentées).

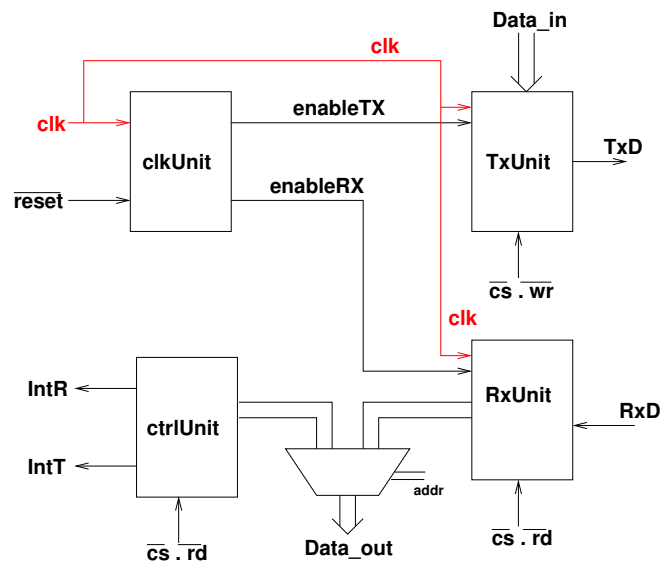


FIGURE 2 – Spécification interne de l'UART

Fonctionnement de l'UART :

- Lorsque le processeur (CPU) reçoit l'interruption `IntT = 0`, l'UART est prête à émettre. Le processeur positionnera `cs = 0` et `wr = 0` pour émettre un caractère donné à l'entrée `data_in` ;
- Lorsque le CPU reçoit l'interruption `IntR = 0`, un caractère a totalement été reçu par l'UART. Pour le récupérer, le CPU positionne `cs = 0` et `rd = 0` ainsi que `addr = 00` afin de lire le registre de réception de l'unité `RxUnit`. Le caractère reçu est placé sur `data_out`.
- À tout moment, le CPU peut connaître l'état de l'UART en lisant son registre de contrôle : `addr = 01`, `cs = 0` et `rd = 0`. La valeur du registre de contrôle sera placée sur `data_out`.

3 L'unité de gestion des “horloges”

3.1 Interface

```
entity clkUnit is
  port ( clk , reset : in  std_logic ;
         enableTX    : out std_logic ;
         enableRX    : out std_logic );
end clkUnit ;
```

3.2 Spécifications

Cette unité a pour objectif de découper la fréquence de l'horloge de base `clk` (celle du processeur) en deux signaux qui vont contrôler les unités d'émission et de réception :

- pour l'émission, `enableTX`, de fréquence 9.6 kHz ;
- pour la réception, `enableRX`, de fréquence 155 kHz.

On fait l'hypothèse que l'horloge en entrée de l'UART `clk` est de fréquence 1550 kHz.

Le signal `enableRX` pour la réception est donc 10 fois plus lent que l'horloge `clk`.

Le signal `enableTX` pour la réception est donc 160 fois plus lent que l'horloge `clk`.

De cette manière, il sera possible à l'unité de réception de contrôler l'asynchronisme de la communication¹.

Ce comportement est donné par la FIGURE 3. Le curseur jaune et les curseurs bleus indiquent bien que les fronts montants de `enableTX` et `enableRX` sont respectivement tous les 160 et 10 cycles de `clk` (période de 10 ns).

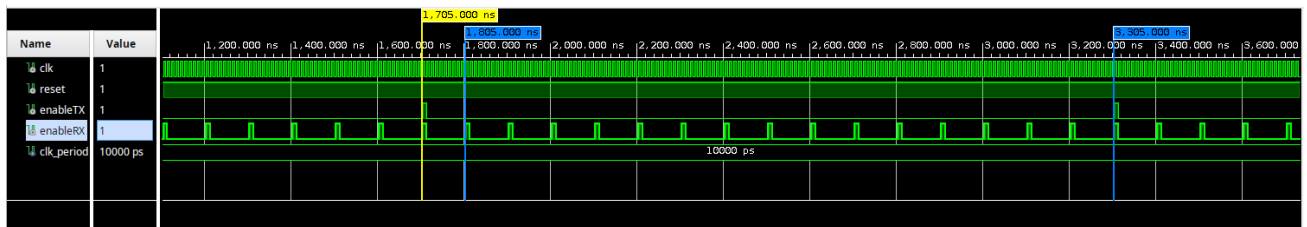


FIGURE 3 – Chronogramme de l'unité `clkUnit`

1. voir transparents de présentation

4 L'unité d'émission

4.1 Interface

```
entity TXUnit is
  port ( clk, reset : in std_logic;
        enable      : in std_logic;
        ld          : in std_logic;
        txd         : out std_logic;
        regE        : out std_logic;
        bufE        : out std_logic;
        data        : in std_logic_vector(7 downto 0) );
end TXUnit;
```

4.2 Spécifications

Cette unité transmet les données `data` bit à bit sur le support de communication via la sortie `txd`.

Le format utilisé est de type RS232 avec 1 bit de stop et sans parité.

L'unité d'émission utilise 2 registres : un registre tampon et un registre d'émission (respectivement notés `BufferT` et `RegisterT`, T pour *Transmission*). L'état de ces registres est donné par les sorties `bufE` et `regE` (E pour *Enable*) :

- `bufE = 1` : le tampon est vide; `bufE = 0` : le tampon est occupé;
- `regE = 1` : le registre d'émission est vide; `regE = 0` : le registre est occupé.

Initialement, `txd = 1`, i.e. la ligne au repos est à l'état haut.

L'émission se déroule en 5 étapes :

1. Attente d'une demande d'émission : lorsque le processeur a une donnée à transmettre, il positionne l'entrée `data` et active l'entrée `ld` (= 1). La donnée est alors chargée dans le tampon;
2. On charge le contenu du tampon dans le registre d'émission;
3. On lance l'émission par l'envoi du bit de start : `txd = 0` ;
4. On émet les 8 bits de données contenues dans le registre d'émission du poids fort au poids faible ;
5. L'émission se termine par le bit de stop : `txd = 1` (équivalent à remettre la ligne au repos).

Le signal `enable` contrôle l'émission : à chaque fois front montant de `clk`, quand le signal `enable` est à 1, on envoie un bit.

Lors d'une transmission, si le tampon est vide, le "processeur" peut demander à émettre une seconde donnée. Ainsi, dès que la transmission en cours est terminée, le TxUnit recommence à partir du point 2 pour transmettre cette autre donnée.

La FIGURE 4 montre un exemple d’émission du caractère 'U' (code ASCII : 0x55).

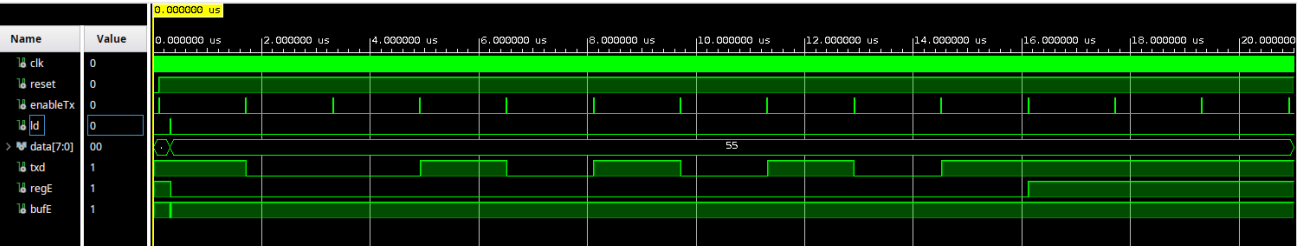


FIGURE 4 – Transmission du caractère 'U'

La FIGURE 5 montre un exemple d’émission du caractère 'U' puis 'E' (0x55 et 0x45) avec le second caractère donné lors de la transmission du 'U'.

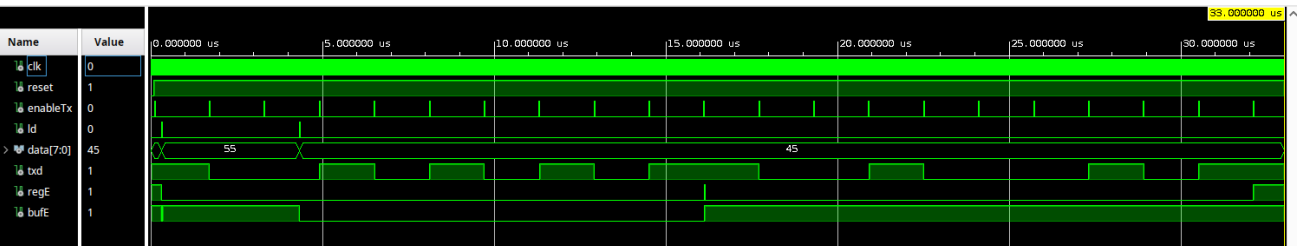


FIGURE 5 – Transmission des caractères 'U' et 'E'

5 L'unité de réception

5.1 Interface

```
entity RxUnit is
  port ( clk, reset      : in  std_logic;
        enable          : in  std_logic;
        read            : in  std_logic;
        rxd             : in  std_logic;
        data            : out std_logic_vector(7 downto 0);
        FErr, OErr, DRdy : out std_logic );
end RxUnit;
```

5.2 Spécifications

Cette unité réceptionne les données sur l'entrée **rxd** et les transmet au processeur via la sortie **data** lorsque la réception est terminée et que le processeur positionne le signal **read**.

Cette unité fournit aussi 3 signaux :

- **DRdy** = 1 lorsqu'une donnée est reçue,
- **FErr** = 1 si la trame reçue est erronée. En l'absence de bit de parité, il n'est seulement possible de détecter que le bit de stop est erroné,
- **OErr** = 1 si lorsqu'une donnée est prête, elle n'est pas lue à temps par le processeur.

Le fonctionnement de l'unité de réception peut être divisé en 2 éléments :

- un élément qui gère les instants de réception;
- un élément qui gère les états de l'unité de réception.

Le premier élément est cadencé par l'horloge du circuit **clk** mais ne travaille que lorsque le signal **enable** est égale à '1'.

Il va d'abord attendre le bit de start.

Quand celui-ci est détecté, cet élément va commencer à compter afin de générer le signal **tmpClk** qui va nous permettre de nous recalculer sur l'"horloge" d'émission. Ce signal **tmpClk** aura ses fronts montants "au milieu" des bits reçus.

Le principe, pour générer **tmpClk**, est donc de compter 8 tops de **enable** dès que l'on détecte le bit de start puis de compter 16, autant de fois qu'il le faut, jusqu'à la réception complète de la trame.

À chaque fois que l'on positionne **tmpClk** à '1', on copie le bit **RxD** de la trame dans **tmpRxD** qui est transmis au second élément.

Le rôle du deuxième élément, toujours cadencé par **clk** est de construire la donnée complète en récupérant le bit **tmpRxD** à chaque fois que **tmpClk** = 1 et de vérifier sa bonne réception; la trame reçue doit être composée :

- du bit de start à '0';
- des 8 bits de données, du poids fort au poids faible;
- du bit de stop à '1'.

À la fin de la réception de la trame, plusieurs cas peuvent se présenter :

1. Si le bit de stop est erroné, i.e. égal à zéro, on avertit le processeur en positionnant $FErr = 1$. La réception est finie.
2. Sinon, on positionne $DRdy = 1$ et la donnée en sortie,
3. On attend au maximum 5 cycles de clk pour que le processeur lise la donnée en positionnant $read = 1$:
 - Si le processeur lit la donnée dans ce délai, tout s'est bien passé; on rabaisse $DRdy$. La réception est finie.
 - Sinon, il y a une erreur de lecture : on positionne $OErr = 1$ et on rabaisse $DRdy$. La réception est finie.

La FIGURE 6 représente les composants de la partie récepteur.

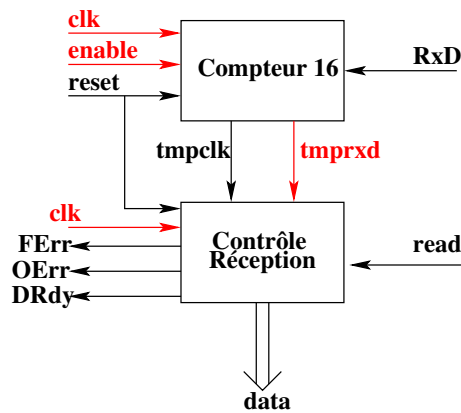


FIGURE 6 – Schéma du récepteur

La FIGURE 7 montre la réception normale du caractère 'U' (Ox55), la FIGURE 8 montre une erreur de transmission et l'émission du signal $FErr$ et la FIGURE 9 un non-positionnement du signal de lecture et l'émission du signal $OErr$.

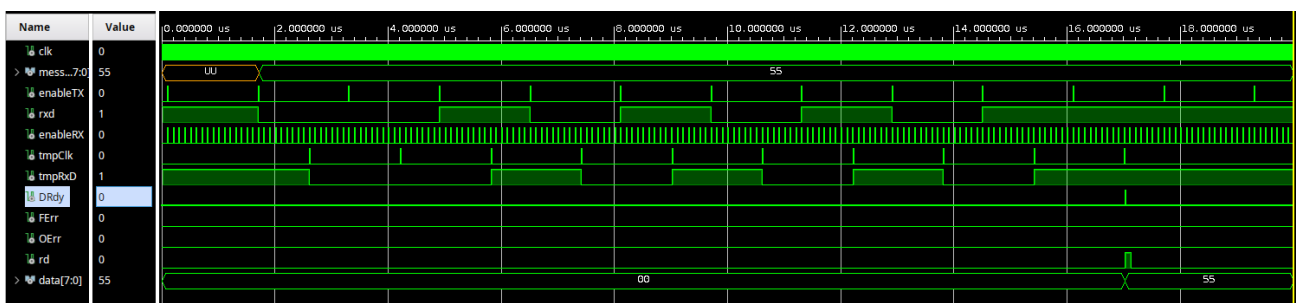


FIGURE 7 – Exemple de la réception normale

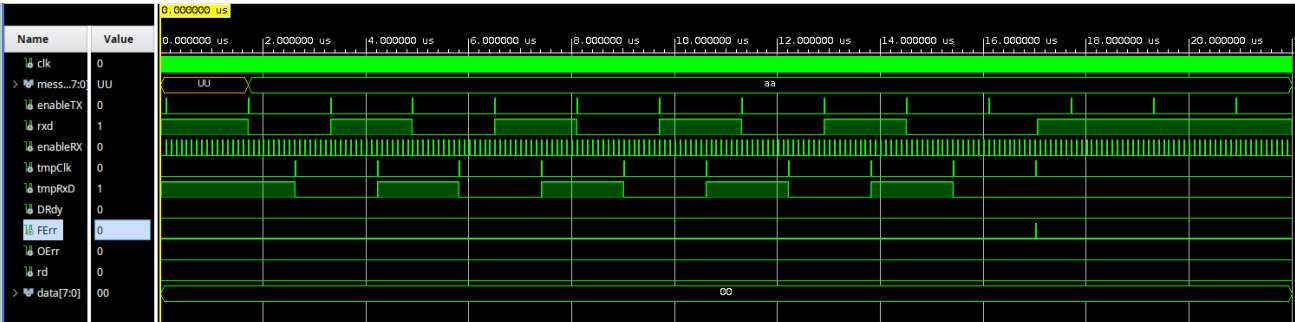


FIGURE 8 – Exemple de réception erronée : bit de stop faux

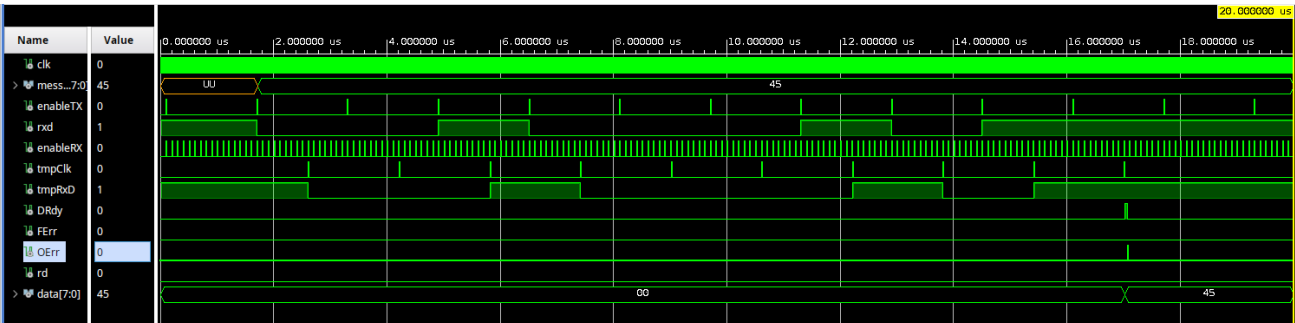


FIGURE 9 – Exemple de non lecture par le processeur : il ne positionne pas read

6 L'unité de contrôle (fournie)

6.1 Interface

```
entity ctrlUnit is
  port ( clk , reset      : in  std_logic ;
         rd , cs          : in  std_logic ;
         DRdy , FErr , OErr : in  std_logic ;
         BufE , RegE      : in  std_logic ;
         IntR             : out std_logic ;
         IntT             : out std_logic ;
         ctrlReg          : out std_logic_vector(7 downto 0) );
end ctrlUnit ;
```

6.2 Spécifications

L'unité de contrôle fournit les signaux d'interruption **IntR** et **IntT** indiquant respectivement si une donnée a été reçue ou non et s'il est possible de transmettre une donnée. Elle gère aussi les données contenues dans le registre **ctrlReg** donné FIGURE 10.

Si une donnée est disponible, **DRdy** = 1, **IntR** = 0. **IntR** sera remis à 1 lorsque la lecture de la donnée est effectuée.

Lorsque le buffer ou le registre d'émission est libre (**BufE** = 1 ou **RegE** = 1), il est possible de transmettre une donnée : **IntT** = 0. Sinon, l'écriture d'une donnée dans le buffer ou le registre d'émission désactive l'interruption : **IntT** = 1.

Gestion du registre de contrôle **ctrlReg** :

- le champs **TxRdy** indique que l'émetteur est prêt à émettre une donnée ;
- le champs **RxRdy** indique que le récepteur a reçu une donnée ;
- les champs **FErr** et **OErr** correspondent aux erreurs de transmission obtenues par l'unité de réception ;
- les champs réservés, notés **Res**, sont toujours à 1.

Res	Res	Res	Res	TxRdy	RxRdy	FErr	OErr
------------	------------	------------	------------	--------------	--------------	-------------	-------------

FIGURE 10 – Le registre de contrôle **regCtrl**.